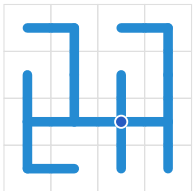
Pipes & Light Up

Giải thuật tìm kiếm cho hai bài toán Logic Puzzle

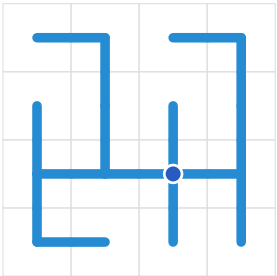


Nhập môn Trí tuệ nhân tạo — BTL1 (HK1 2026–2027)

DFS (mù) + Greedy Best-First (Pipes) · DFS + MRV/CP (Light Up)

[Nhóm — thành viên điền tại đây]

Mục tiêu: một mạng liên thông, không chu trình



- Mọi cổng mở đều gặp cổng tương ứng của ô kề.
- Không cổng nào chìa ra ngoài bàn chơi.
- Tất cả các ô liên thông. Không có chu trình.

Ô là đỉnh,connections là cạnh:

$$\text{G là cây} \implies |E| = n - 1$$

Nguồn xanh chỉ để trực quan hóa nước chảy.

Luật: puzzle-pipes.com. Kiểm tra đích: model.py và search.py.

Phát biểu bài toán tìm kiếm

Input	$n = m^2$ ô, hình dạng cố định, biên thường (không wrap)
Trạng thái	Tiền tố $s = (a_0, \ldots, a_{k-1})$ các định hướng đã gán
Trạng thái đầu	Tiền tố rỗng $()$, biết trước mọi hình dạng
Tác động	Gán một định hướng cho ô k theo thứ tự row-major
Chuyển tiếp	$s' = s + (a_k)$, nhận sau khi qua kiểm tra khả thi
Đích	$k = n$ và bàn hoàn chỉnh đạt mọi luật
Chi phí	1 mỗi phép gán; không giải thuật nào dùng để xếp hạng

Một bước solver có thể tương ứng nhiều lần xoay 90°.
Mọi lời giải hợp lệ đều chấp nhận; không tối ưu số lần xoay.

Mô hình trạng thái cài đặt: Search trong pipes_problem/search.py.

Các successor hợp lệ

Ô kế tiếp có hình dạng cố định; tìm kiếm chọn định hướng của nó.



Ví dụ ô góc trên-trái. Biên trái và trên cấm hướng N và W.

Khớp láng giềng cũng kiểm tra cả cổng không có.
Ứng viên phải qua hết các lớp cắt tỉa dùng chung trước khi được nhận.

Ví dụ minh họa. Lọc định hướng: boundary_domains() và compatible().

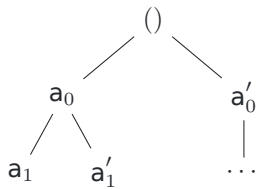
Không gian tìm kiếm

Số xoay phân biệt mỗi ô

Khuỷu, chữ T, đầu cụt	4
Ống thẳng	2
Chữ thập	1

$$U = \prod_{i=0}^{n-1} |R_i| \leq 4^n$$

Cây tiền tố



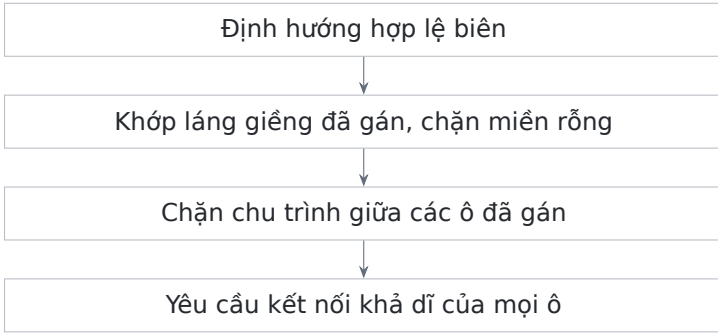
Độ sâu n, hệ số phân nhánh tối đa 4.
Mỗi tiền tố có đúng một cha.

Số cách gán đầy đủ, trước kiểm tra biên.

Ví dụ 4×4 kèm theo: $U = 268{,}435{,}456$.
Đây là cận trên số cách gán, không phải số node đo được.

Đếm từ các xoay phân biệt trong pipes-4x4-261.json.

Cắt tỉa dùng chung



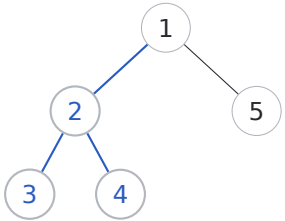
Cả hai giải thuật dùng cùng các kiểm tra và cùng thứ tự ô.

Điều kiện gốc: tổng số cổng $= 2(\mathrm{n} - 1)$.
Kết nối khả dĩ là điều kiện cần, chưa chứng minh tồn tại nghiệm.

Cài đặt cắt tỉa: Search.compatible(). Phân tích đầy đủ: docs/pipes-analysis.md.

Tìm kiếm mù: Depth-First Search

```
VISIT(s):
  if budget exhausted: return LIMIT
  count expansion
  if GOAL(s): return SOLVED(s)
  for a in rotations of tile |s|:
    child = s + (a)
    if FEASIBLE(child):
      r = VISIT(child)
      if r is SOLVED or LIMIT: return r
  return UNSOLVABLE
```



Thứ tự thăm minh họa.
Đi sâu một nhánh rồi quay lui.

Đầy đủ trên cây hữu hạn với đủ tài nguyên và cắt tỉa hợp lệ.
Trường hợp xấu nhất vẫn hàm mũ.

Giả mã rút gọn khớp dfs(). Kiểm tra gốc chạy trước VISIT.

Heuristic của Greedy: số lựa chọn còn lại

$$h(s) = \sum_{\text{i chưa gán}} \log_2 |D_i(s)|$$

$D_i(s)$ là miền định hướng hợp lệ biên, khớp láng giềng đã gán.

Ứng viên	Kích thước miền còn lại	h(s)
A	$(1, 2, 4)$	$0 + 1 + 2 = 3$
B	$(1, 1, 2)$	$0 + 0 + 1 = 1$

Greedy ưu tiên B: giá trị nhỏ hơn nghĩa là ít lựa chọn còn lại.
Thứ tự hòa: tiền tố sâu hơn, rồi tới chèn sớm hơn.

Điểm số bỏ qua ràng buộc giữa các ô chưa gán.
Không admissible, không đảm bảo tối ưu số lần xoay.

Ví dụ miền giả định. Độ ưu tiên thật: (h, -depth, thứ tự chèn) trong greedy().

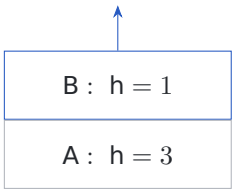
Greedy Best-First search

```
push () into min-priority queue
while queue is not empty:
  if budget exhausted: return LIMIT
  s = pop minimum-priority state
  count expansion
  if GOAL(s): return SOLVED(s)
  for a in rotations of tile |s|:
    child = s + (a)
    D = FEASIBLE(child)
    if D exists:
      push child with (h(child),
                       -|child|, serial)
return UNSOLVABLE
```

Kiểm tra gốc chạy trước khi push. Độ ưu tiên chỉ dùng h.

Chọn từ hàng đợi

mở rộng tiếp



Mọi tiền tố khả thi đều nằm trong hàng đợi.

Bộ nhớ có thể tăng hàm mũ.

Giao thức đo lường

- Cùng 9 map cho cả hai giải thuật.
- Ba map mỗi kích thước, mỗi giải thuật chạy 5 lượt.
- Cùng cắt tỉa, cùng thứ tự ô, cùng giới hạn tài nguyên.



Thời gian không gồm vẽ hình và phát lại. Bộ nhớ là đỉnh cấp phát Python được trace. Chỉ gộp hai lần đo khi kết quả trùng khớp.

Giới hạn: 30 giây tính toán và 200,000 lần mở rộng mỗi lượt.

Chạy batch: benchmark.py. Đo: measure() và worker.py.

Kết quả thực nghiệm

Size	Algorithm	Solved/runs	Time (ms)	Peak (KiB)
4×4	DFS	15/15	3.149	26.984
	Greedy	15/15	3.097	12.359
5×5	DFS	15/15	5.261	53.297
	Greedy	15/15	6.157	19.219
7×7	DFS	15/15	24.469	145.852
	Greedy	15/15	31.695	27.594

Median trên 15 lượt thành công mỗi kích thước và giải thuật.
Mỗi nhóm gồm 3 input phân biệt, 5 lượt bấm giờ.

DFS nhanh hơn ở 5×5 và 2/3 map 7×7; Greedy thắng trên map khó nhất (38.2 so với 40.8 ms) và giữ bộ nhớ traced thấp hơn 4–5 lần.
Input này khá dễ với cắt tỉa hiện có, chưa đủ để kết luận kẻ thắng chung.

Đo ngày 25/10/2026, Linux, Python 3.13. Dữ liệu thô: docs/benchmarks/2026-10-25-pipes.csv.

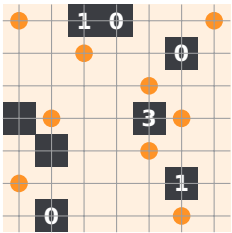
Phạm vi và thí nghiệm tiếp theo

- Thêm map khó hơn và map vô nghiệm đã chứng minh.
- Đo riêng đóng góp từng lớp cắt tỉa (ablation).
- Giữ tách biệt UNSOLVABLE với LIMIT/CANCELLED.
- Thử heuristic khác (MRV) và mở rộng bộ input.

Tài liệu tham khảo

Luật Pipes: `https://www.puzzle-pipes.com/`
Russell & Norvig, Artificial Intelligence: A Modern Approach, 4th ed., chương 3 và 6. `https://aima.cs.berkeley.edu/`

Bài toán 2: Light Up (Akari)



- Chiếu mọi ô trắng; ánh sáng dừng tại tường hoặc biên.
- Hai bóng không được nhìn thấy nhau.
- Tường số có đúng số bóng kề trực tiếp yêu cầu.
- So **DFS** với **Backtracking + MRV + CP** để tìm nghiệm đầu tiên.

Bàn 7×7 easy, seed 70097 hiện tại; nghiệm CSP đã kiểm chứng.

Generator: walls-first-witness-v2. A* vẫn có trong app nhưng không đo trong đợt này.

Phát biểu bài toán và đối tượng so sánh

- Trạng thái: mask bóng W bit, mask sáng, mask vị trí còn hợp lệ.
- Ban đầu chưa có bóng; đích: đủ sáng, đúng tường số, không xung đột.
- DFS: ô tối đầu tiên, thử row-major, chấp nhận rồi đi sâu ngay.
- CSP: propagation đến điểm cố định, MRV động, ưu tiên lợi ích chiếu sáng.
- Cùng bàn và ngân sách; khác cắt tỉa và công việc trong mỗi node.

Cả DFS lẫn CSP không bảo đảm tối ưu số bóng.
Đo hiệu ứng kết hợp MRV/CP, không tách riêng tác dụng của MRV.

Cài đặt: light_up_problem/algorithms/search.py.

DFS: nhận bóng hợp lệ đầu rồi đi sâu

```
DFS(state):
  check budgets
  if GOAL(state): return SOLVED
  target = first dark cell, row-major
  for bulb in legal illuminators(target):
    if FEASIBLE(place(bulb)):
      search child immediately
      if solved or limited: return result
    exclude bulb from later siblings
  return UNSOLVABLE
```

Frame tường minh giữ ứng viên chưa thử; liên kết cha dựng lại đường đi. Chỉ quay lui khi cây con thất bại. Không gian xấu nhất tăng hàm mũ.

depth_first_search(); TIMEOUT / NODE_LIMIT / MEMORY_LIMIT khác UNSOLVABLE.

MRV chọn ràng buộc; CP suy ra quyết định bắt buộc

- Tường còn u ô hợp lệ, thiếu r bóng: $\binom{u}{r}$ lựa chọn.
- Ô tối: một lựa chọn mỗi nguồn chiếu hợp lệ. Chọn miền nhỏ nhất; hòa ưu tiên tường; thử ứng viên chiếu nhiều ô tối trước.
- CP: tường đủ bóng $\Rightarrow$ cấm các ô kề còn lại.
- Số bóng thiếu bằng số ô kề hợp lệ $\Rightarrow$ buộc đặt tất cả.
- Một nguồn chiếu duy nhất $\Rightarrow$ buộc đặt bóng, loại các đường nhìn.
- Suy luận đường chéo loại vị trí làm tường thiếu ô kề hợp lệ.

Lặp đến khi không đổi; có mâu thuẫn thì loại nhánh.

_View.select_mrv_choices() và _View.propagate().

Backtracking + MRV + Constraint Propagation

```
CSP(state):
  check budgets; propagate to fixed point
  if contradiction: reject branch
  if GOAL(state): return SOLVED
  constraint = minimum remaining choices
  for choice ordered by lighting gain:
    apply choice; propagate
    if consistent: descend immediately
    if solved or limited: return result
    restore parent checkpoint
  return UNSOLVABLE
```

Checkpoint phục hồi cả bóng và miền hợp lệ. Một node CSP có thể gồm nhiều bóng bắt buộc, nên số node không đại diện lượng công việc tương đương.

backtracking_mrv_cp(); stack tường minh tránh giới hạn đệ quy Python.

Input: năm seed khác nhau cho mỗi cấu hình

- Chọn tường trước; đặt witness trên ô tối ngẫu nhiên đến khi đủ sáng.
- Suy ra số tường; khoảng 25% tường không số; kiểm chứng witness độc lập.
- Cỡ 7/10/14/25 $\times$ easy/normal/hard $\times$ 5 seed = **60 bàn**.
- Mật độ tường: 16% / 10.5% / 5.5%; chưa kiểm tra duy nhất nghiệm.
- Hai giải thuật nhận đúng cùng bàn ở mỗi seed.

Seed = cỡ $\times$ 10.000 + chỉ số độ khó $\times$ 97 + offset (0–4).
Chỉ số độ khó là 1/2/3. Năm bàn khác nhau, không lặp năm lần một bàn.

JSON, seed và SHA-256 đầy đủ: 2026-10-31-lightup-dfs-csp-manifest.json.

Demo trực quan

- DFS / A* / CSP (MRV + CP), Start Search, Pause, Resume, hai mũi tên.
- Đủ lịch sử ở **2 / 8 / 30 bước/giây**; không bỏ bước.
- CSP: từng bóng bắt buộc và lần thu hẹp miền, cả propagation ban đầu.
- Dấu $\times$ trên ô tối là ô cấm, tách khỏi ghi chú chơi tay.
- Quay lui phục hồi bóng và miền; kết quả cuối chờ hiện xong trace.

Worker spawn giữ UI phản hồi. Visualizer không timeout, vẫn giới hạn node/bộ nhớ. Benchmark giữ đủ ba giới hạn.

ui/app.py + worker.py. Nút Compare trong app vẫn so DFS/A*.

Giao thức: 120 lượt, giữ nguyên giới hạn

- 60 bàn × DFS/CSP; một lượt mỗi giải thuật mỗi seed; process mới, tuần tự.
- Đảo thứ tự giải thuật ở seed liên tiếp. Không trace/animation.
- Cùng **30 giây / 500.000 node / 512 MiB bộ nhớ ước lượng**.
- Thời gian solver khi bật tracemalloc; đỉnh cấp phát Python, không phải RSS.
- Không tính sinh bàn/khởi động process; kiểm chứng mọi kết quả SOLVED.
- Ghi riêng `TIMEOUT`, `NODE_LIMIT`, `MEMORY_LIMIT`.

Lượt bị dừng không phải giải xong hay bằng chứng vô nghiệm.
Median bảng gồm mọi lượt thử; median riêng lượt SOLVED có trong JSON summary.

31/10/2026; CSV + summary + manifest môi trường/mã nguồn/input trong docs/benchmarks/.

Kết quả: 7 / 10

	DFS			CSP (MRV + CP)		
Cỡ / mức	S/5	s	KiB	S/5	s	KiB
7 easy	5/5	0.0075	24.9	5/5	0.0066	25.2
7 normal	5/5	0.0066	26.8	5/5	0.0103	29.0
7 hard	5/5	0.0074	27.3	5/5	0.0107	30.5
10 easy	5/5	0.0229	55.9	5/5	0.0288	56.2
10 normal	5/5	0.0218	60.0	5/5	0.0309	61.8
10 hard	5/5	0.0229	62.4	5/5	0.0314	67.8

S: lượt SOLVED. Median trên 5 seed, gồm cả thời lượng lượt bị dừng.
KiB là đỉnh cấp phát Python, không phải RSS hay bộ nhớ ước lượng.

31/10/2026; 2026-10-31-lightup-dfs-csp.csv; 30 s / 500,000 nodes / 512 MiB.

Kết quả: 14 / 25

		DFS			CSP (MRV + CP)	
Cỡ / mức	S/5	s	KiB	S/5	s	KiB
14 easy	5/5	0.2330	135.3	5/5	0.0801	109.1
14 normal	5/5	0.0724	117.3	5/5	0.1030	123.5
14 hard	5/5	0.0832	126.1	5/5	0.1168	134.6
25 easy	2/5	30.0001	2237.0	5/5	0.9558	562.1
25 normal	1/5	30.0004	3583.3	5/5	1.0274	696.9
25 hard	4/5	0.7835	870.3	5/5	1.0421	868.0

S: lượt SOLVED. Median trên 5 seed, gồm cả thời lượng lượt bị dừng.
KiB là đỉnh cấp phát Python, không phải RSS hay bộ nhớ ước lượng.

31/10/2026; 2026-10-31-lightup-dfs-csp.csv; 30 s / 500,000 nodes / 512 MiB.

Diễn giải kết quả đo

- Giải được: **DFS 52/60**, **CSP 60/60**; TIMEOUT: 8 / 0.
- NODE_LIMIT / MEMORY_LIMIT: đều **0 lượt** ở cả hai; vẫn giữ các giới hạn.
- Trong 52 cặp cùng giải xong, CSP nhanh hơn ở 15; median $t_{DFS}/t_{CSP} = 0.73$. Không gồm cặp bị dừng.
- Bàn nhỏ cắt tỉa tốt có thể lợi cho DFS; CP tốn thêm chi phí mỗi node.
- CSP giảm rẽ nhánh; một node có thể gồm nhiều bóng suy ra bắt buộc.

Năm seed là mẫu nhỏ; mật độ tường không xếp hạng được độ khó giải đơn điệu.

Số liệu mới DFS/CSP trên generator hiện tại; không trộn dữ liệu DFS/A* ngày 25/10.

Hạn chế và thực nghiệm tiếp theo

- Không khẳng định tối ưu số bóng hoặc nghiệm duy nhất.
- Tracemalloc gây overhead, không đo RSS.
- Năm seed mỗi cấu hình: cần tăng mẫu và bổ sung bàn vô nghiệm.
- Ablation CP/MRV, đo thời gian riêng khi tắt tracemalloc.

Luật: `https://www.puzzle-light-up.com/`
Russell & Norvig, Artificial Intelligence: A Modern Approach, Ch. 3 và 6.
Cài đặt và phân tích: `docs/lightup-analysis.md`.

Tổng kết

- Pipes: DFS / Greedy. Light Up đo mới: DFS / MRV + CP.
- Demo đủ bước tìm kiếm; CSP có cả từng suy luận bắt buộc.
- Light Up: 60 bàn khác nhau, 120 lượt theo cặp, giữ nguyên giới hạn.
- Ghi tỷ lệ giải được và TIMEOUT / NODE_LIMIT / MEMORY_LIMIT riêng.
- Dữ liệu: `docs/benchmarks/2026-10-31-lightup-dfs-csp.csv`.

Tài liệu tham khảo

`https://www.puzzle-pipes.com/` · `https://www.puzzle-light-up.com/`
Russell & Norvig, Artificial Intelligence: A Modern Approach, 4th ed.
Cài đặt và phân tích: `pipes_problem/`, `light_up_problem/`, `docs/`.